

Travaux Pratiques : Listes Chaînées (TP1 ASD2)

Institut Supérieur des Technologies de l'Information et de la Communication (ISTIC) — Correction

Exercice 1

Question : Définir une structure Etudiant contenant : id (entier), nom (chaîne de caractères), moyenne (réel).

Correction :

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int id;
    char nom[50];
    float moyenne;
} Etudiant;

int main() {
    Etudiant e1;
    return 0;
}
```

Exercice 2

Question : Écrire une fonction qui permet de fusionner deux listes chaînées d'entiers triées par ordre croissant en une troisième liste, telle que celle-ci contienne les éléments de la liste 1 et les éléments de la liste 2, triés par ordre croissant.

Correction :

```
void fusionner(list l1, list l2, list *l3) {
    list *prec, *n;
    if ((l1 == NULL) && (l2 == NULL)) {
        *l3 = NULL;
    } else {
        n = malloc(sizeof(list));
        *l3 = n;
        while ((l1 != NULL) && (l2 != NULL)) {
            if ((*l1).suivant < (*l2).suivant) {
                (*n).x = (*l1).x;
                l1 = (*l1).suivant;
            } else {
                (*n).x = (*l2).x;
                l2 = (*l2).suivant;
            }
            (*n).suivant = malloc(sizeof(list));
            prec = n;
            n = (*n).suivant;
        }
        if (l1 != NULL) {
            l1 = l2; // (selon la fiche de lecture)
            while (l1 != NULL) {
                (*n).x = (*l1).x;
                l1 = (*l1).suivant;
                (*n).suivant = malloc(sizeof(list));
                prec = n;
                n = (*n).suivant;
            }
        }
        free(n);
        (*prec).suivant = NULL;
    }
}
```

Exercice 3

1) Créez une liste chaînée qui comprend trois éléments (données 1, 2, 3). La structure comprend deux champs : donnée et un pointeur suivant.

2) Affichez les données de la liste.

Correction :

```
#include <stdio.h>
#include <stdlib.h>

typedef struct element *list;
struct element {
    int donnee;
    list suivant;
};

int main() {
    list l1, n, p;
    int i;
    l1 = NULL;
    for (i = 1; i <= 3; i++) {
        n = malloc(sizeof(struct element));
        n->donnee = i;
        n->suitant = l1;
        l1 = n;
    }
    p = l1;
    while (p != NULL) {
        printf("      %d", p->donnee);
        p = p->suitant;
    }
    return 0;
}
```

Exercice 4

Structure de base pour une liste chaînée double :

```
typedef struct element *list;
struct element {
    list *prec;
    int donnee;
    list suiv;
};
```

1) Procédure qui accepte une suite d'entiers différents de -1 et les charge dans la liste chaînée L dans l'ordre inverse de leur introduction (chargeinverse).

```
void chargeinverse(list *tete, list *queue) {
    int x;
    list n;
    *tete = NULL;
    *queue = NULL;
    printf("donner un entier :
");
    scanf("%d", &x);
    while (x != -1) {
        n = malloc(sizeof(struct element));
        n->donnee = x;
        n->prec = NULL;
        if (*tete == NULL) {
            n->suiv = NULL;
            *tete = n;
            *queue = n;
        } else {
            n->suiv = *tete;
            (*tete)->prec = n;
            *tete = n;
        }
        printf("donner un entier :
");
        scanf("%d", &x);
    }
}
```

2) Procédure qui accepte une suite d'entiers différents de -1 et les charge dans la liste chaînée L dans le même ordre de leur introduction (chargequeue).

```
void chargequeue(list *tete, list *queue) {
    int x;
    list n;
    *tete = NULL;
    *queue = NULL;
    printf("donner un entier :
");
    scanf("%d", &x);
    while (x != -1) {
        n = malloc(sizeof(struct element));
        n->donnee = x;
        n->prec = NULL;
        if (*tete == NULL) {
            n->prec = NULL;
            *tete = n;
        }
    }
}
```

```

        *queue = n;
    } else {
        n->prec = *queue;
        (*queue)->suiv = n;
        *queue = n;
    }
    printf("donner un entier :
");
    scanf("%d", &x);
}
}

```

3) Procédure qui permet de supprimer un entier x dans une liste chaînée double d'entiers (supprime & recherche).

```

list recherche(list tete, list queue, int x) {
    list p, v = NULL;
    int t = 0;
    p = tete;
    while (p != NULL && t == 0) {
        if (p->donnee == x) {
            t = 1;
        } else {
            p = p->suiv;
        }
    }
    if (t == 1) {
        v = p;
    }
    return v;
}

void supprime(list *tete, list *queue, int x) {
    list e, pos;
    pos = recherche(*tete, *queue, x);
    if (pos != NULL) {
        if (pos == *tete) {
            *tete = (*tete)->suiv;
            if (*tete == NULL) {
                *queue = NULL;
            } else {
                (*tete)->prec = NULL;
            }
        } else {
            e = pos->prec;
            e->suiv = pos->suiv;
            if (pos != *queue) {
                (pos->suiv)->prec = e;
            } else {
                *queue = e;
            }
        }
        free(pos);
    }
}

```